

Sorting (Again !)

Types

- In-place
- Stable

Why branching in code is slower?

```
int Smaller(int a, int b)
{
    if(a < b)
        return a;
    else
        return b;
}
```

50%

50%

Solution without branching

```
int Smaller_Branchless(int a, int b)
{
    return a*(a<b)
        +b*(b<=a);
}
```

Solution without branching

```
void ToUpperCPP(char* d, int count)
{
    for (int i = 0; i < count; i++)
    {
        if (d[i] >= 'a' && d[i] <= 'z')
            d[i] -= 32;
    }
}
```

Solution without branching

```
void ToUpper_Branchless2(char* d, int count)
{
    for (int i = 0; i < count; i++)
    {
        d[i] -= 32 * (d[i] >= 'a' && d[i] <= 'z');
    }
}
```

Counting Sort

Unsorted
Array

4	9	3	8	4	9	6	5	3	8	5	0	4	9	9	0	4	4	5	2	6	4	8	0	8	9	5	1	7	9	4	5
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Counters

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

Counting Sort

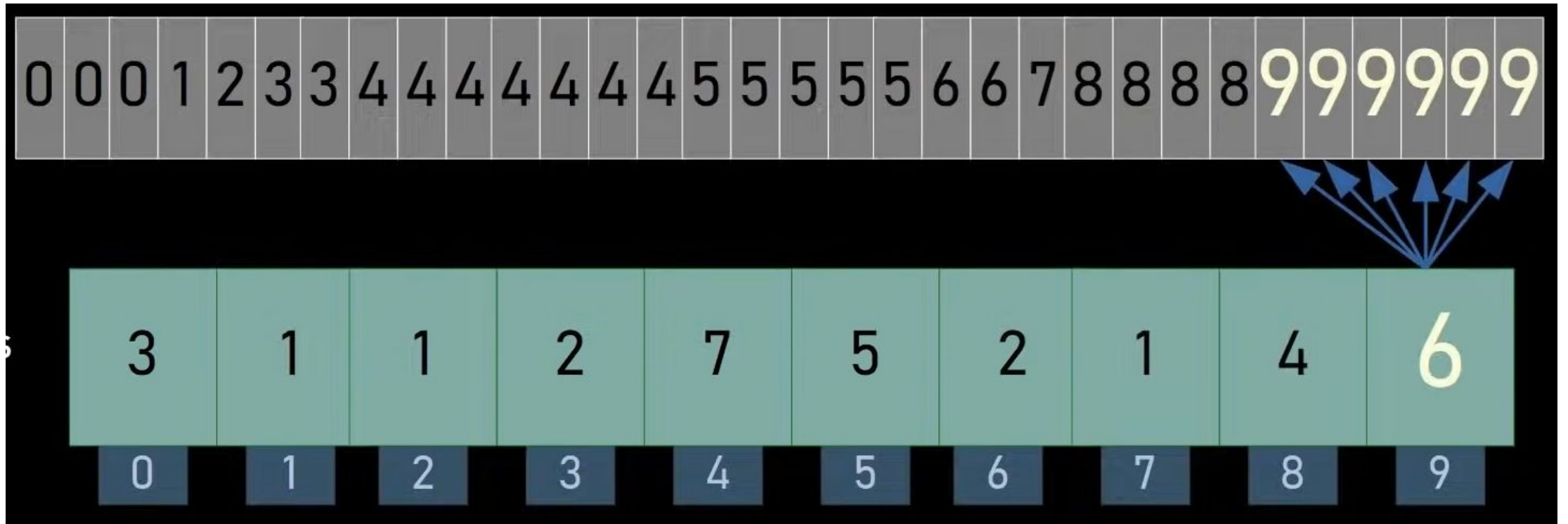
Unsorted
Array

4	9	3	8	4	9	6	5	3	8	5	0	4	9	9	0	4	4	5	2	6	4	8	0	8	9	5	1	7	9	4	5
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Counters

3	1	1	2	7	5	2	1	4	6
0	1	2	3	4	5	6	7	8	9

Counting Sort



Run Time complexity?

- Run through data array once
- Run through counting array, and overwrite data into original array
- $N+N$
- $O(N)$

Limitations

- Works better only for unsigned integers. (Signed integers and other data types will need special handling of the data. Sometimes it may be impossible)
- Need a lot of memory for counting array for bigger values.

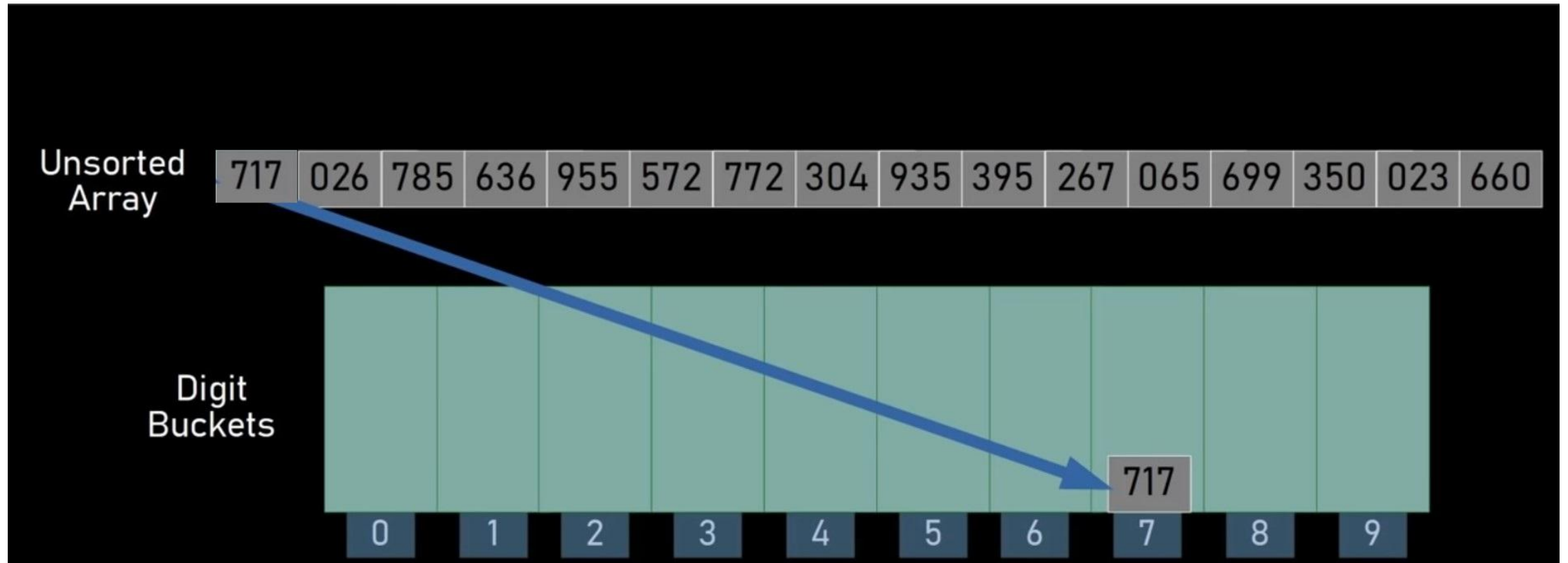
Memory requirement

- For a 32-bit integers, you will need a counter array of size 4,294,967,296.
- Which requires $((4294967296/1024/1024/1024) = 4\text{GB}$ of RAM. (If each counter uses only 1 Byte).
- If counter is integer type array, then each counter needs 4 Bytes. In that case 16GB RAM will be needed.
- For a 64-bit integer, the counter array will need
 $2^{64} * 8 / 1024 / 1024 / 1024 = 137,438,953,472 \text{ GB}$ (138 Exabytes)
- World's total memory capacity was approximately, 55(year 2000), 295(year 2007) Exabytes.

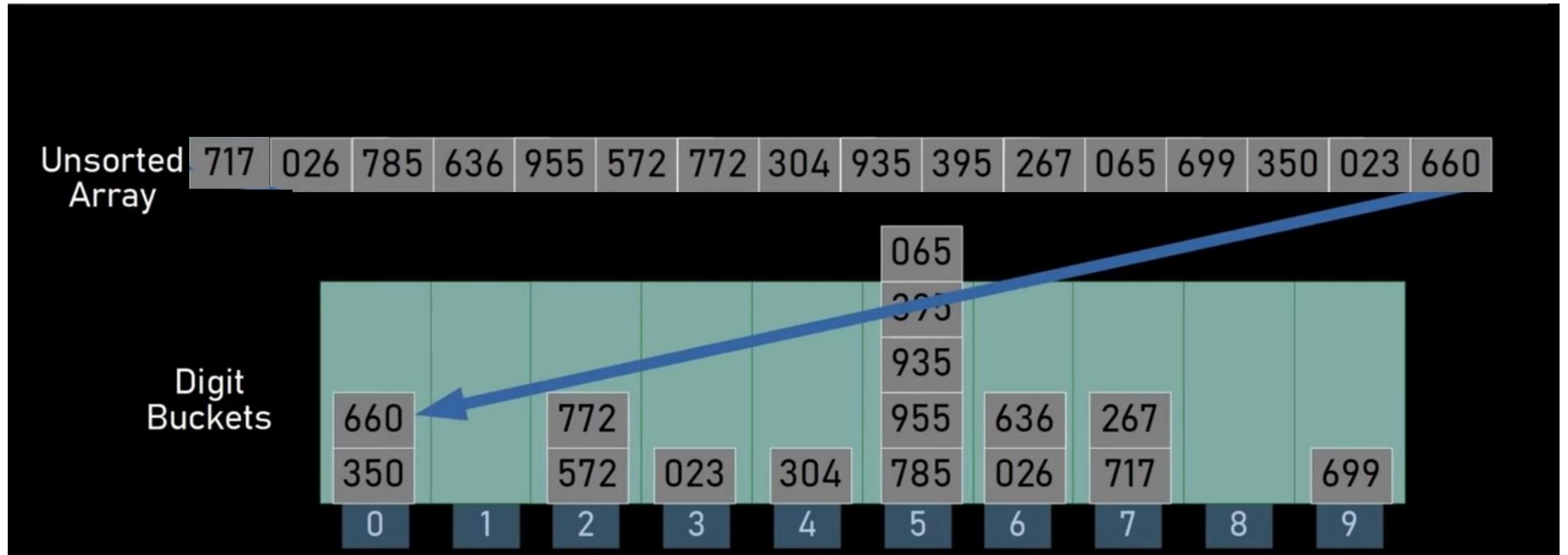
Alternatives?

- Trade speed for space.
- Radix Sort using bucket
- Radix Sort using counter

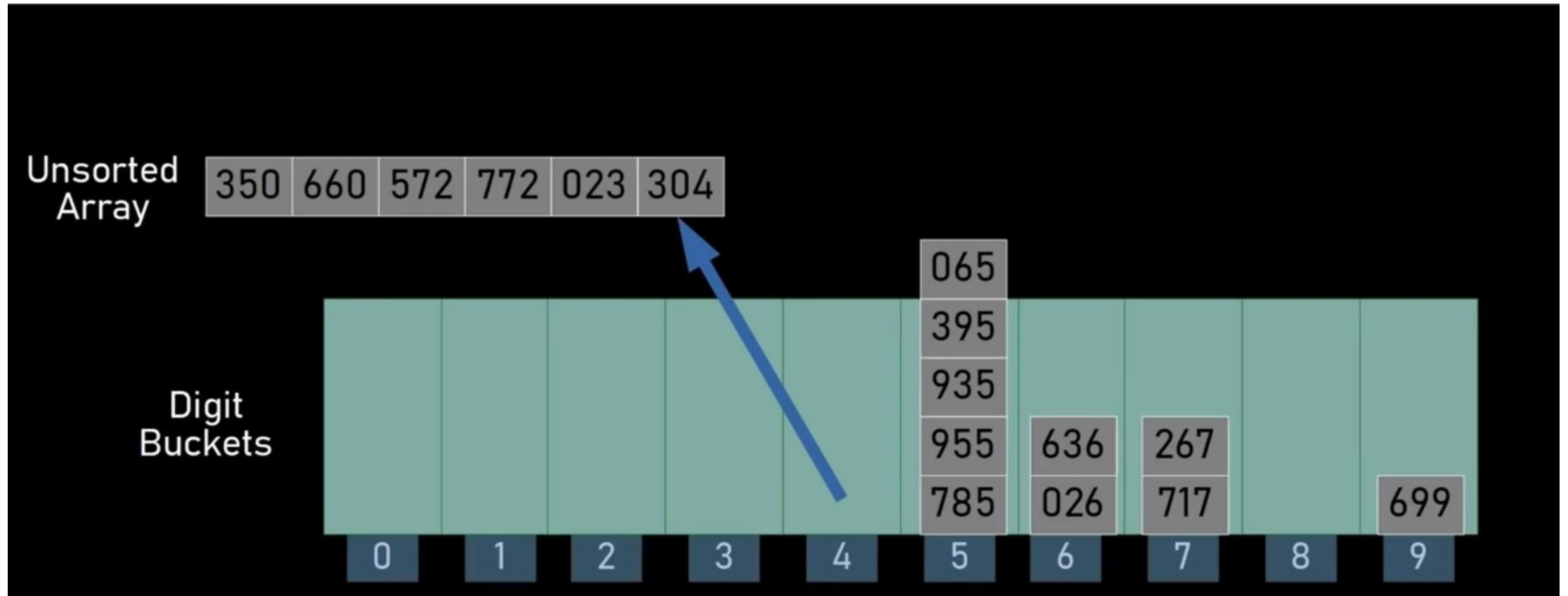
Radix sort (using bucket)



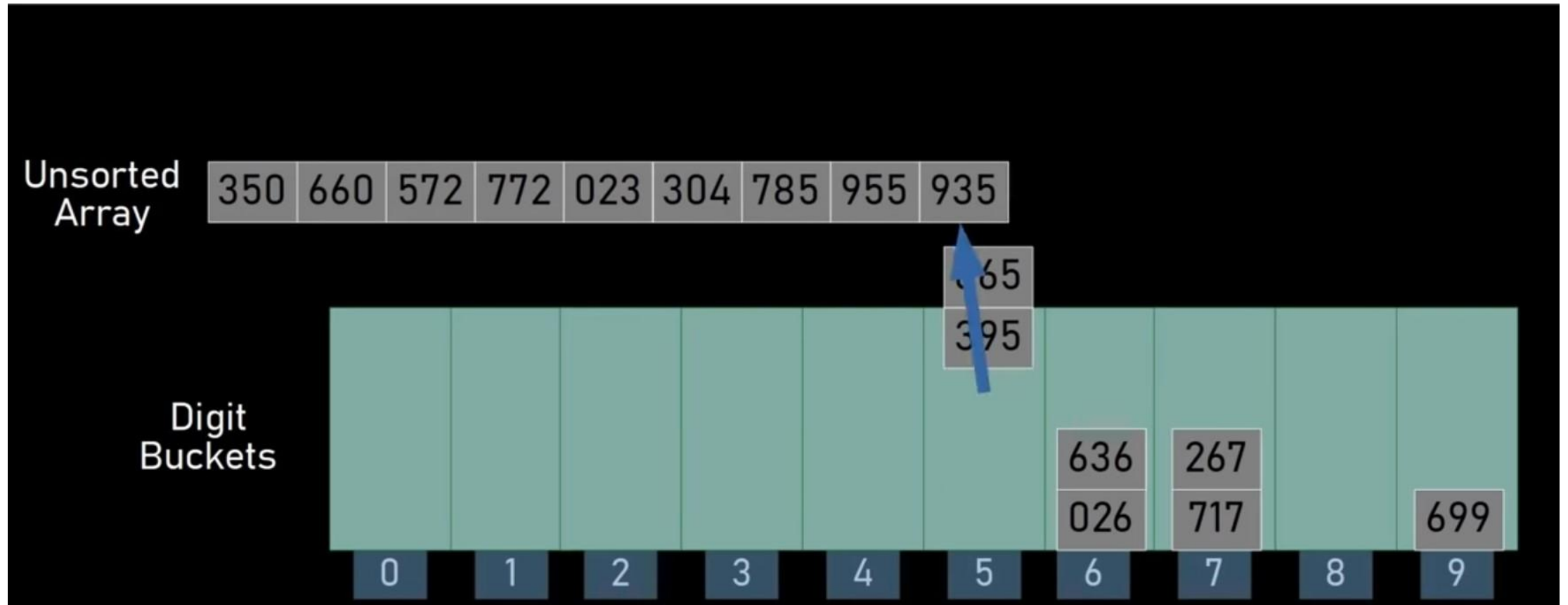
Radix sort (using bucket)



Radix sort (using bucket)



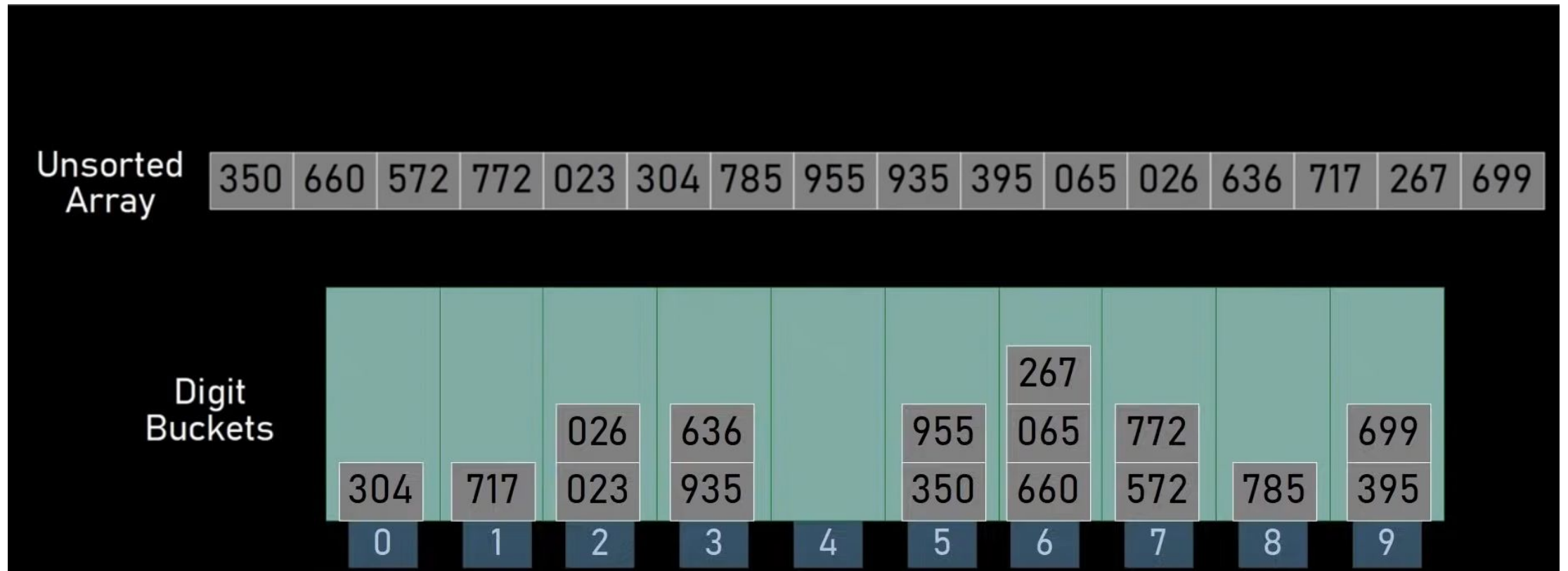
Radix sort (using bucket)



Radix sort (using bucket)



Radix sort (using bucket)

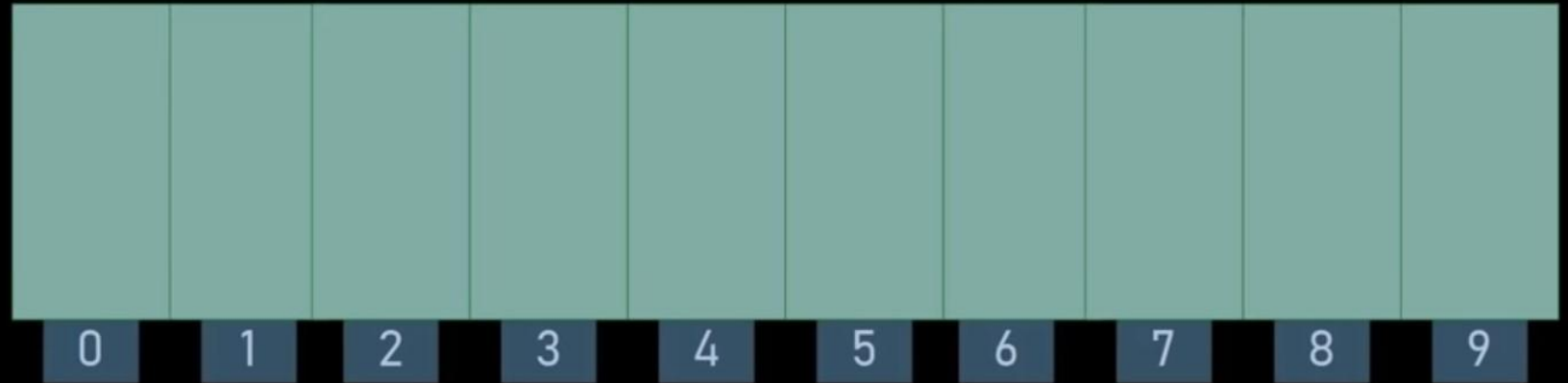


Radix sort (using bucket)

Unsorted
Array

304	717	023	026	935	636	350	955	660	065	267	572	772	785	395	699
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Digit
Buckets



Radix sort (using bucket)

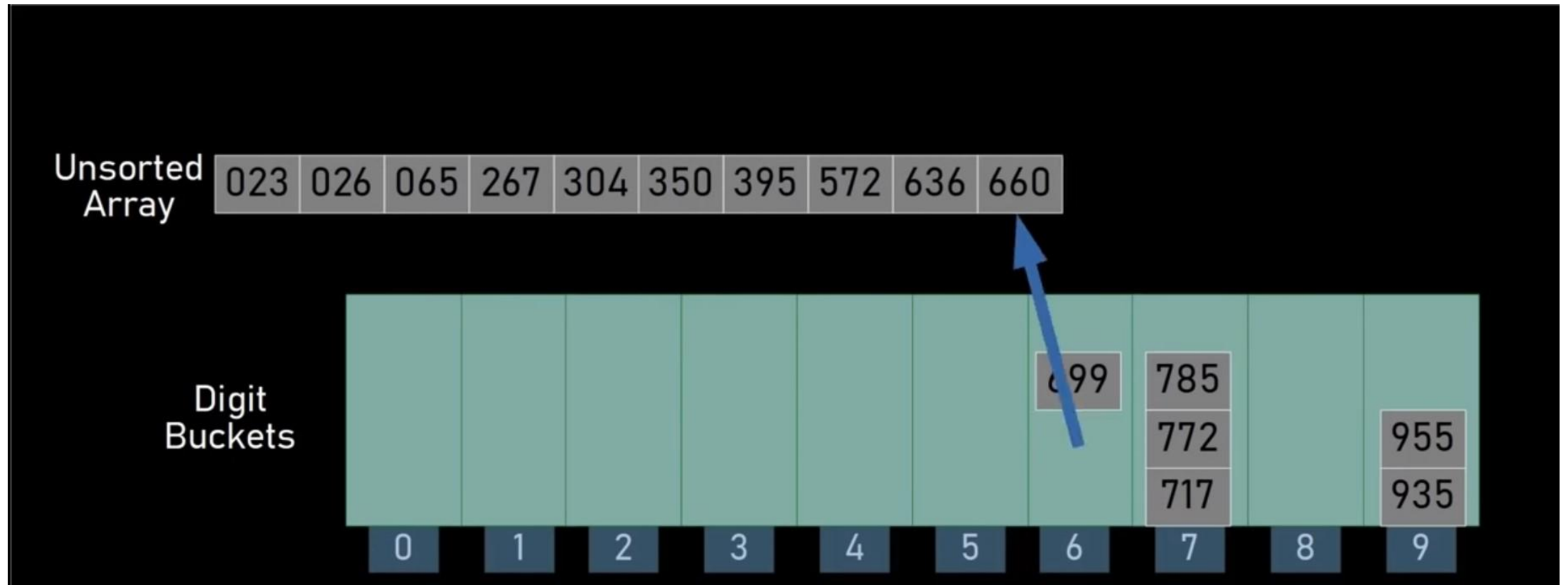
Unsorted
Array

304	717	023	026	935	636	350	955	660	065	267	572	772	785	395	699
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Digit
Buckets



Radix sort (using bucket)



Radix sort (using bucket)



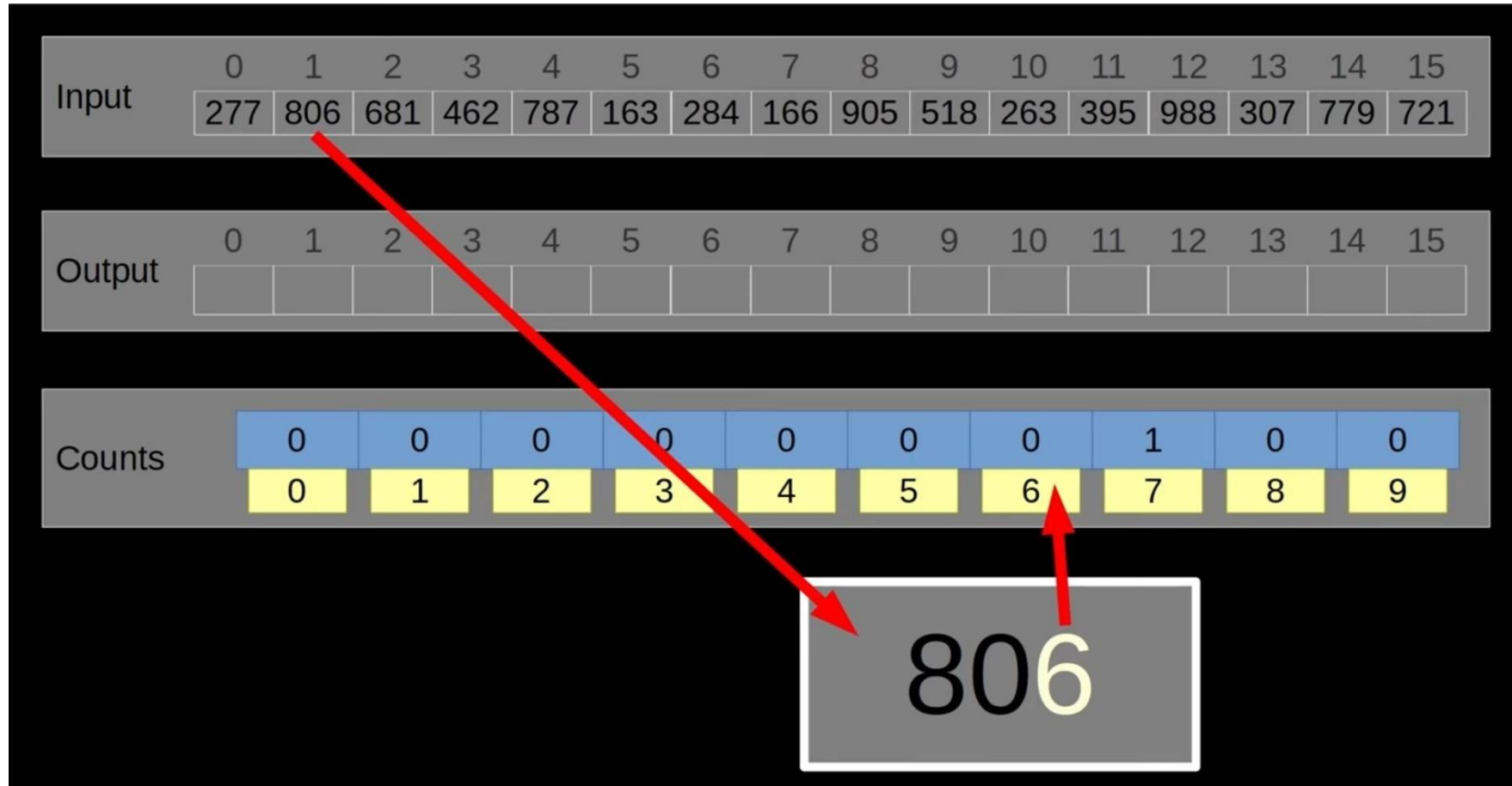
Limitations

- How do we manage buckets? (There is no bucket in the RAM!)
- A Queue or linked list can be used.
- But linked list needs a lot of allocation and de-allocation of memory.
- It also needs additional space for pointer along with each elements

Radix Sort Using Counters

Input	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	277	806	681	462	787	163	284	166	905	518	263	395	988	307	779	721
Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Counts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9						

Radix Sort Using Counters



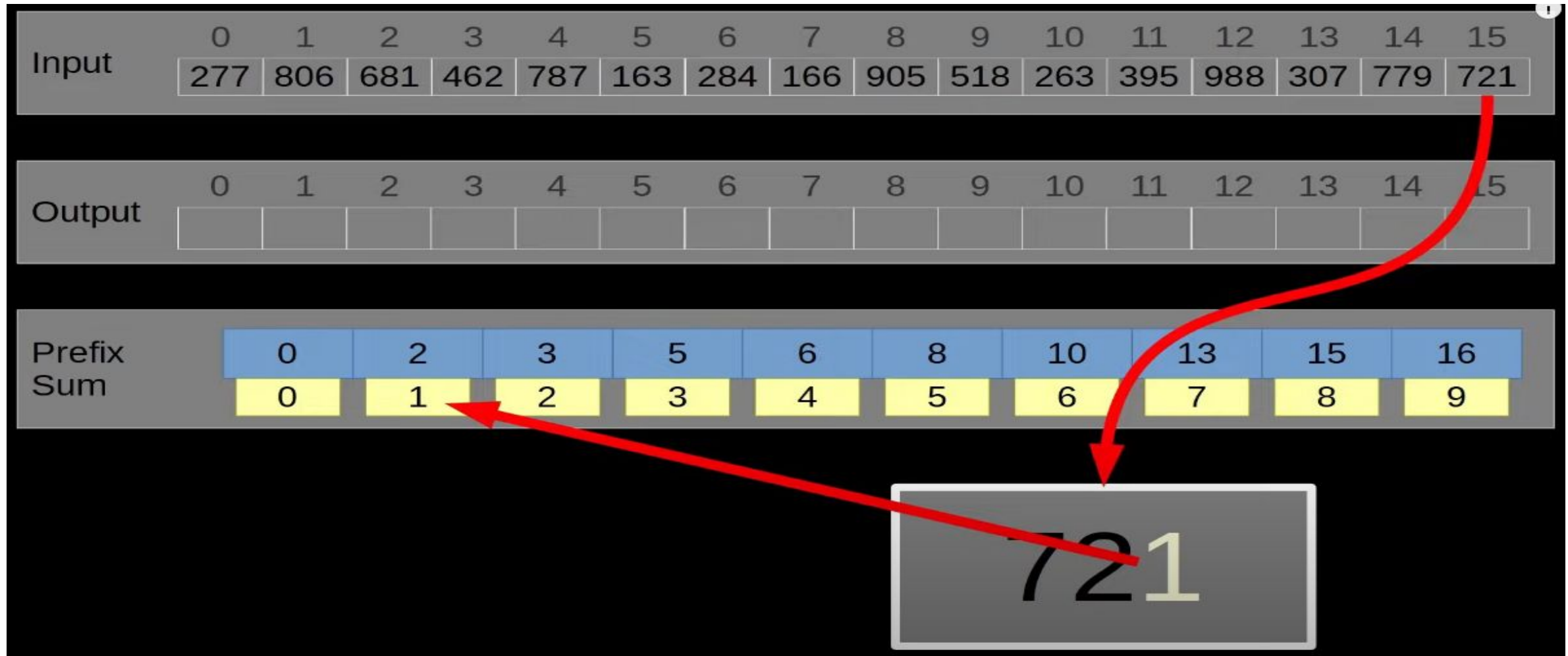
Radix Sort Using Counters

Input	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	277	806	681	462	787	163	284	166	905	518	263	395	988	307	779	721
Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Counts	0	2	1	2	1	2	2	3	2	1						
	0	1	2	3	4	5	6	7	8	9						

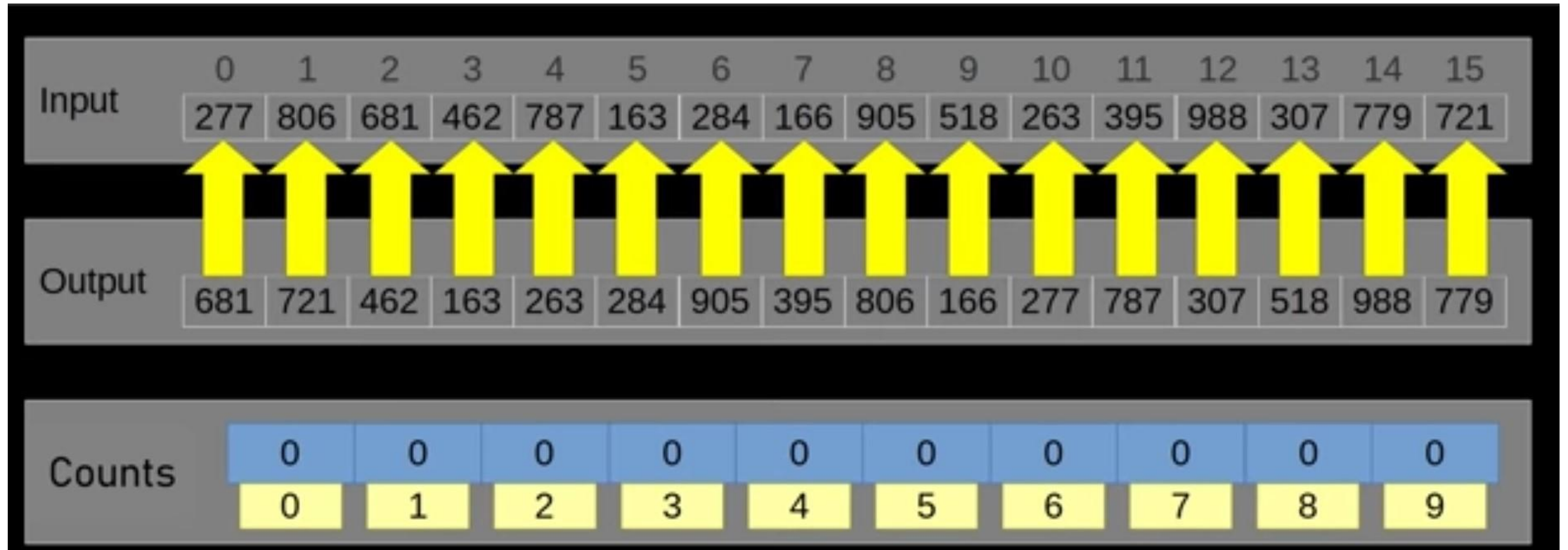
Radix Sort Using Counters

Input	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	277	806	681	462	787	163	284	166	905	518	263	395	988	307	779	721
Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Prefix Sum	0	2	3	5	6	8	10	13	15	16						
	0	1	2	3	4	5	6	7	8	9						

Radix Sort Using Counters



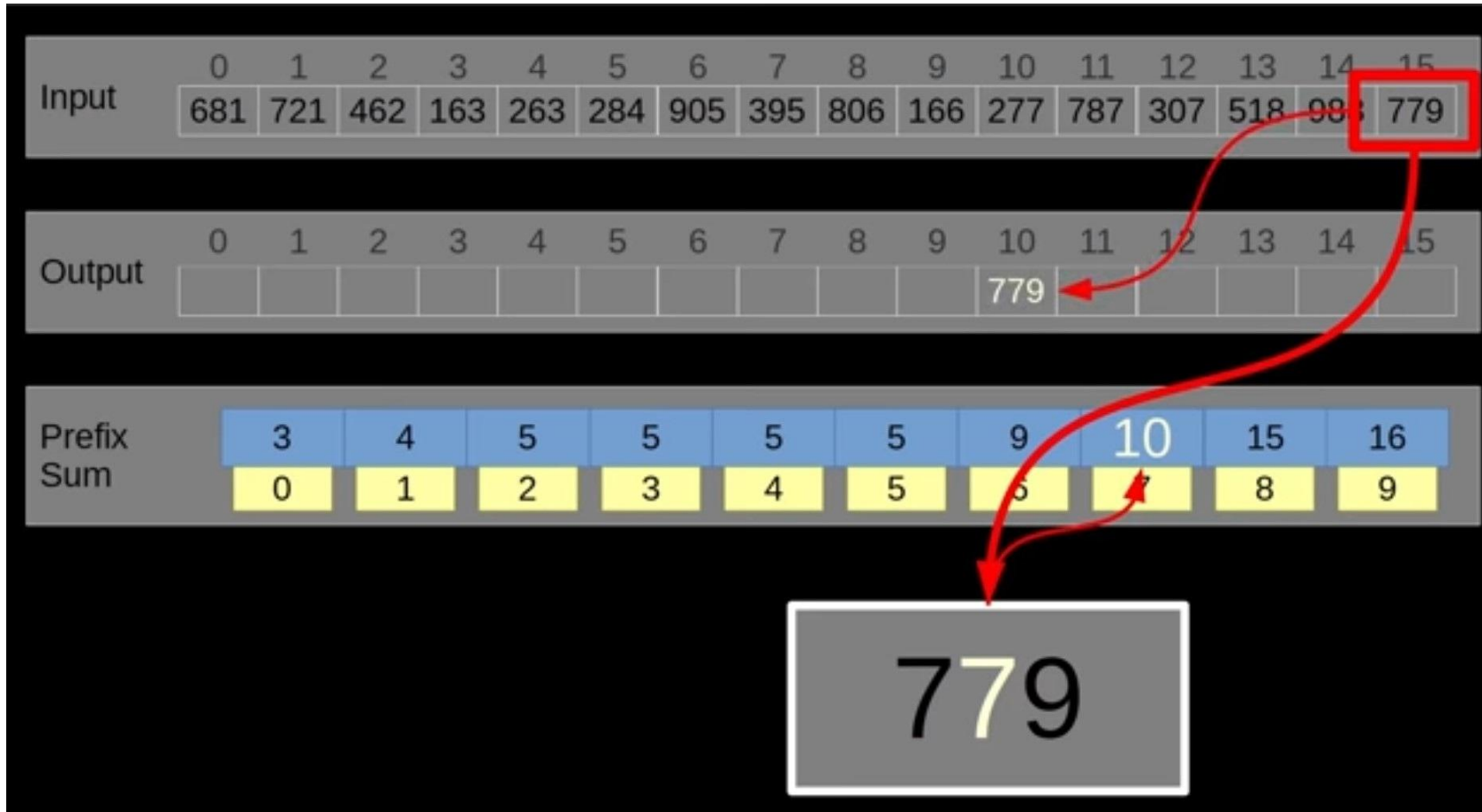
Radix Sort Using Counters



Radix Sort Using Counters

Input	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	681	721	462	163	263	284	905	395	806	166	277	787	307	518	988	779
Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Counts	3	1	1	0	0	0	4	2	4	1						
	0	1	2	3	4	5	6	7	8	9						

Radix Sort Using Counters



Radix Sort Using Counters

Input	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	905	806	307	518	721	462	163	263	166	277	779	681	284	787	988	395
Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Counts	0	2	3	2	1	1	1	3	1	2						
	0	1	2	3	4	5	6	7	8	9						

Radix Sort Using Counters

Input	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	905	806	307	518	721	462	163	263	166	277	779	681	284	787	988	395
Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	163	166	263	277	284	307	395	462	518	681	721	779	787	806	905	988
Prefix sum	0	0	2	5	7	8	9	10	13	14						
	0	1	2	3	4	5	6	7	8	9						

Runtime Complexity

- $O(N)$